

README

token_optimizer

Revision: 2 **Last modified:** 2026-07-08T07:18:45Z **Status:** active **Description:** Project-agnostic Go engine that minimizes the token / cost / byte footprint of LLM request pipelines — tier routing with a never-downgrade floor, a multi-layer cache, shape-routed wire encoding, and telemetry — fully decoupled from any consuming project.

What it is

`token_optimizer` is a standalone, reusable Go module (`module github.com/vasic-digital/token_optimizer`) that sits in front of an LLM request path and reduces its token, dollar, and byte cost. It is the engine behind the ATM-659 token-reduction research (workstreams WS1–WS11); this repository is the production cut of that design.

The request path is one binary, fast, and single-flight-safe across a large context fleet. Planned packages (this repo scaffolds them incrementally):

Package	Responsibility
<code>pkg/config</code>	Decoupling surface — runtime registry of tiers, pricing, thresholds, alternatives, and the never-downgrade predicate. Implemented.
<code>pkg/pipeline</code>	<code>Optimize(ctx, Request)</code> orchestrator binding cache → router → wire → tier → telemetry.
<code>pkg/router</code>	Tier decision + never-downgrade HARD floor + failover.
<code>pkg/cache</code>	Exact / semantic / artifact cache layers + invalidation.
<code>pkg/wire</code>	Shape-routed encoder — <code>min(TOON, compactJSON)</code> , never-worse guard.
<code>pkg/trans-port</code>	HTTP/3 + brotli seam (binary never on the model-token channel).
<code>pkg/tier</code>	Tier endpoint adapters (deterministic / local / alias / native), runtime-registered.
<code>pkg/telemetry</code>	JSONL decision spine + price-table load + p95 reporting.

Full architecture: the consuming project's `docs/research/tokens/IMPLEMENTATION_ARCHITECTURE.md` .

Decoupling contract

The engine ships **zero project constants**. It never hardcodes a tier name, an endpoint, a price, a threshold, or a task-taxonomy value. Every project-specific datum is registered **at runtime** by the consumer through a `*config.Config` . Consequently the request path is byte-identical whether the module is vendored, referenced in-tree, or reused — only the startup registration differs.

There is exactly one coupling seam: `pkg/config` . If you find a project-specific string anywhere else in this repository, it is a bug.

How a consumer registers its data

```

import "github.com/vasic-digital/token_optimizer/pkg/config"

func buildConfig() (*config.Config, error) {
    c := config.New() // seeded with config.DefaultNeverDowngrade

    // 1. Register the completion tiers this project uses (cheapest / most
    // deterministic first via Priority).
    if err := c.RegisterTier(config.Tier{
        Name: "T-DET", Priority: 0, Deterministic: true, // free, shells to a util
    }); err != nil {
        return nil, err
    }
    if err := c.RegisterTier(config.Tier{
        Name: "T-LOCAL-8B", Priority: 10,
        Endpoint: "http://127.0.0.1:18434/v1/chat/completions", // price stays 0
    }); err != nil {
        return nil, err
    }
    if err := c.RegisterTier(config.Tier{
        Name: "T-NATIVE", Priority: 30,
        PricePerMTokIn: 3, PricePerMTokOut: 15,
    }); err != nil {
        return nil, err
    }

    // 2. Register failover alternatives (used when a primary endpoint is down).
    if err := c.RegisterAlternative("T-LOCAL-8B", "T-NATIVE"); err != nil {
        return nil, err
    }

    // 3. Register routing thresholds (consumer-defined keys).
    c.SetThreshold("semantic_cosine_floor", 0.86)

    // 4. Optionally replace the load-bearing never-downgrade floor.
    // The default forbids routing a load-bearing request to a cheaper tier.
    c.SetNeverDowngrade(func(current, candidate config.Tier, loadBearing bool) bool {
        if !loadBearing {
            return false
        }
        return candidate.CombinedPrice() < current.CombinedPrice()
    })

    return c, nil
}

```

The engine reads tiers, alternatives, thresholds, and the predicate from this `*config.Config` — it never assumes any specific value is present.

Building & testing

```
go build ./...  
go test ./...
```

Dependencies

Own-org dependencies are declared in `helix-deps.yaml` and are resolved from the parent project root (never as nested own-org submodules).

License

[MIT](#).